



API SYSTEMS

Part 3 – Evaluation



MONICA ARONG DARMANIN
INTERACTIVE DIGITAL MEDIA

Part 3 – Evaluation

Introduction

During this unit, I explored APIs and how they operate in real-world applications. This project involved developing an API and was tested using Postman to evaluate its functionality and performance, which helped identify errors and ensure endpoints functioned correctly.

This project focused on the learning management system that was developed in the previous semester. This involved extending and testing an API to manage student data, ensuring it functions correctly with the existing database structure.

Security and Principles

Security considerations were researched using the OWASP API Security Top 10, which highlighted common security risks and vulnerabilities in APIs. Researching these risks informed how security was incorporated into the API, including areas such as authentication and access control. From this, I understood risks such as broken authentication, injection attacks and insecure data exposure. Using these principles reduced the risk of unauthorised access and increased the safeguarding of confidential student data.

I implemented Broken Object Level Authorisation, as it is one major risk to the system, and this happens when users have access to data that is not theirs. This ensured that users could only access their own records, improving the security and reliability of the API.

I also applied Broken Authentication, which allows attackers to gain access to users' accounts. To reduce this risk, my API would ensure that it is secure with OAuth 2.0, to ensure secure authentication and expiring tokens. Additionally, Broken Object Property Level Authorisation was considered to enforce property-level permission, allowing the fields only for authorised users. This prevents the exposure or manipulation of sensitive data such as administrative data or restricted student data.

API Design and Structure

Separating the various components of the project (e.g., database connection files, endpoint files, and class files) into their own folders improved the organisation and navigation more efficiently during development.

Since I was working on the previous website developed in semester one, the database structure was already developed with the proper information, data types, constraints, and relationships. I utilised the different HTTP request methods allowed the API to execute multiple operations effectively, such as retrieving, updating, and deleting data.

Testing and Debugging

Testing each endpoint using Postman helped ensure correct functionality and checked the database to confirm data was stored correctly. I was able to quickly see patterns for errors occurring at the various endpoints. This process improved my ability to debug requests and ensure that identical requests produced consistent API responses. By comparing test cases from multiple versions of Postman, I was able to refine my endpoints and ensure stable responses across the system.

Testing with cURL

A simple PHP website was built to consume the API using cURL. Using cURL helped demonstrate how external client applications communicate with the API through HTTP requests. This allowed requests and responses to be tested outside of Postman in a more realistic client environment.

The primary interface was created using index.php, which contained an interface to select the different types of requests to the API, and result.php, which contained all of the code to execute each type of request to the API and display results.

The testing process was improved by allowing for API requests to be executed outside of Postman, helping verify that endpoints, authorisation and response worked consistently across the different environments.

HTTP Responses Implementation

The use of HTTP response codes was implemented to improve how the API communicates success and error states between the client and server. For example, the 200 OK response was used to help confirm that the endpoint was functioning correctly during testing in Postman.

Another example, the 400 Bad Request response code was returned when the client sent invalid or incomplete data. The 404 Not Found was used when the requested resource did not exist. This implementation of response codes helps provide feedback and makes it easier to improve the user experience when interacting with the API.

Authentication and Role-Based Access

OAuth 2.0 bearer token authentication was implemented to secure API endpoints by restricting access to authorised users only. Each request required a valid access token in the Authorisation header, which ensured only authenticated users could access protected resources. Role-based access control ensured different user types (Admin, Lecturer, Student) could only access the data relevant to their permissions. Postman environments were added to simulate different user roles, each with its own bearer token.

Challenges Faced

Challenges I encountered were handling inconsistent or invalid data inputs, which caused errors when interacting with the database and affected API reliability. Additionally, I encountered issues with authentication, where invalid or missing bearer tokens were not initially handled correctly, leading to unauthorised access errors. This showed how important secure token handling is in preventing unauthorised access to sensitive data.

These issues were resolved through improved validation, debugging, and refinements to request handling logic, which improved the stability and reliability of the API. This highlighted weaknesses in access control, so additional validation was implemented to ensure that requests without valid tokens were rejected, improving the overall security of the API. This made debugging easier as errors could now be identified more quickly across different endpoints.

Conclusion

Overall, this process demonstrates how APIs are designed, built, and tested in a structured environment through Postman, which made it easier to identify errors and improve endpoints quickly. The project highlighted the importance of organisation, testing and security when handling data to ensure a secure and reliable API.

In future development, I would improve the API by strengthening authentication mechanisms, enhancing error handling, and improving input validation to make the system more robust and secure. By doing this, I can reduce the chances of unauthorised access and improve overall performance and reliability. This project improved my understanding of how APIs manage authentication, validation, and communication between client and server.